Plano de refactor do frontend do Ethos para suportar a migraÃ§Ã£o local-first

Objetivo
Preparar o frontend do Ethos para que o core clÃnico deixe de depender estruturalmente do backend remoto e passe a operar em um modelo **local-first**, com persistÃªncia local como fonte primÃ¡ria de verdade e serviÃ§os online como camada opcional.

Resumo executivo

Hoje o frontend do Ethos mostra alguns sinais claros de acoplamento que dificultam a migraÃ§Ã£o local-first:
- `Index.tsx` concentra responsabilidades demais
- navegaÃ§Ã£o, domÃnio, autenticaÃ§Ã£o e renderizaÃ§Ã£o estÃ£o muito prÃ³ximos
- services parecem assumir API remota como origem principal
- stores misturam estado de interface com preocupaÃ§Ãµes operacionais
- a experiÃªncia jÃ¡ estÃ¡ adaptada a backend remoto e cold start

O refactor recomendado deve acontecer **sem reescrita total**, em camadas, com foco em:
1. modularizaÃ§Ã£o
2. separaÃ§Ã£o de responsabilidades
3. introduÃ§Ã£o da camada local de dados
4. substituiÃ§Ã£o gradual do fluxo API-first por local-first

Objetivos do refactor

Objetivos principais
- reduzir o acoplamento do frontend atual
- criar fronteiras claras entre UI, domÃnio, persistÃªncia e serviÃ§os remotos
- permitir que o frontend consuma dados locais como fonte primÃ¡ria
- desacoplar autenticaÃ§Ã£o online do uso clÃnico principal
- preparar o app para backup, restore e cloud opcional no futuro

Objetivos nÃ£o prioritÃ¡rios nesta fase
- redesign visual completo
- reescrita total de rotas
- mudanÃ§a completa de framework
- expansÃ£o para mobile

DiagnÃ³stico atual

1. `Frontend/src/pages/Index.tsx` estÃ¡ sobrecarregado
Hoje esse arquivo concentra:
- splash e intro
- autenticaÃ§Ã£o
- seleÃ§Ã£o de pÃ¡ginas
- roteamento lÃ³gico interno
- papel do usuÃ¡rio
- prefetch
- renderizaÃ§Ã£o das pÃ¡ginas principais
- controle de sessÃ£o selecionada e paciente selecionado

Problema
Esse padrÃ£o dificulta:
- manutenÃ§Ã£o
- testes
- modularizaÃ§Ã£o por domÃnio
- evoluÃ§Ã£o local-first
- degradaÃ§Ã£o elegante offline/online

2. `App.tsx` e a camada de API carregam lÃ³gica de infraestrutura remota
HÃ¡ lÃ³gica para:
- readiness gate
- wake-up do backend
- keep-alive
- retries
- banner de â€œaguardando servidorâ€

Problema
Isso mostra que o frontend assume o backend remoto como prÃ©-requisito do fluxo principal.

3. Stores ainda nÃ£o refletem o modelo desejado
As stores atuais parecem adequadas para:

- estado de tela
- jobs
- flags de conectividade
- seleÃ§Ã£o atual

Mas nÃ£o devem virar a fonte primÃ¡ria do domÃnio clÃnico.

4. AutenticaÃ§Ã£o estÃ¡ acoplada ao app inteiro
O `AuthContext` atual estÃ¡ muito ligado a:
- token remoto
- restauraÃ§Ã£o de sessÃ£o remota
- validaÃ§Ã£o online

Problema
Isso conflita com a meta de separar:
- desbloqueio local
- conta online opcional

Arquitetura-alvo do frontend

Camadas recomendadas

1. App shell
ResponsÃ¡vel por:
- layout geral
- navegaÃ§Ã£o principal
- estrutura base
- composiÃ§Ã£o de providers

2. Feature modules
Separar o frontend por domÃnio funcional, por exemplo:
- `patients`
- `sessions`
- `records` ou `prontuarios`
- `documents`
- `reports`
- `agenda`
- `auth`
- `account`
- `admin`

Cada mÃ³dulo deve conter:
- pÃ¡ginas
- componentes especÃficos
- hooks do domÃnio
- adapters de leitura/escrita
- integraÃ§Ã£o com repositories

3. Domain services
ResponsÃ¡veis por:
- regras de negÃ³cio
- coordenaÃ§Ã£o entre entidades
- transformaÃ§Ã£o de dados
- orquestraÃ§Ã£o de fluxos

4. Local repositories
ResponsÃ¡veis por:
- leitura e escrita local
- acesso Ã persistÃªncia local
- consultas por domÃnio

5. Remote adapters
ResponsÃ¡veis por:
- comunicaÃ§Ã£o com serviÃ§os online quando necessÃ¡rio
- integraÃ§Ãµes opcionais
- sync futuro
- backup remoto futuro

6. UI state layer
ResponsÃ¡vel apenas por:
- estado transitÃ³rio de interface
- seleÃ§Ã£o atual

- modais
- filtros
- loading local
- feedback visual

PrincÃpios do refactor

1. UI nÃ£o acessa storage diretamente
Componentes e pÃ¡ginas nÃ£o devem ler IndexedDB ou API remota diretamente.

2. ServiÃ§os de domÃnio nÃ£o devem depender da UI
A lÃ³gica central deve ser consumÃvel independentemente da tela.

3. PersistÃªncia local Ã© a fonte primÃ¡ria
Sempre que o domÃnio principal for consultado, o frontend deve priorizar a camada local.

4. Online Ã© complementar
O remoto deve ser tratado como extensÃ£o do fluxo, nÃ£o como sua fundaÃ§Ã£o.

5. Refactor incremental
Cada domÃnio deve poder migrar sem exigir reescrita completa do restante do sistema.

Plano de execuÃ§Ã£o do refactor

Fase 1 â€” Separar fundaÃ§Ã£o do app
Objetivo
Reduzir o acoplamento estrutural do topo da aplicaÃ§Ã£o.

MudanÃ§as
- quebrar responsabilidades de `App.tsx`
- quebrar responsabilidades de `pages/Index.tsx`
- separar:
  - app bootstrap
  - auth bootstrap
  - shell
  - page resolution
  - feature rendering

Resultado esperado
O app deixa de depender de um Ãºnico arquivo como orquestrador principal.

Fase 2 â€” Introduzir mÃ³dulos por domÃnio
Objetivo
Organizar o frontend por fronteiras funcionais reais.

Exemplo de estrutura alvo
- `src/modules/patients`
- `src/modules/sessions`
- `src/modules/records`
- `src/modules/documents`
- `src/modules/reports`
- `src/modules/agenda`
- `src/modules/auth`
- `src/modules/account`

Resultado esperado
Cada Ã¡rea passa a evoluir com menor acoplamento.

Fase 3 â€” Introduzir camada de dados local-first
Objetivo
Criar o caminho tÃ©cnico para consumir dados locais como fonte primÃ¡ria.

MudanÃ§as
- introduzir database adapter local
- introduzir repositories por domÃnio
- introduzir services de domÃnio que dependam de repositories, nÃ£o da API remota

- adaptar pÃ¡ginas crÃticas para consumir hooks baseados em domÃnio local

Resultado esperado
O frontend deixa de ser API-first nas Ã¡reas migradas.

Fase 4 â€” Refactor da autenticaÃ§Ã£o
Objetivo
Separar acesso local e conta online.

MudanÃ§as
- isolar auth online em mÃ³dulo prÃ³prio
- preparar camada de acesso local ao core clÃnico
- impedir que sessÃ£o remota seja prÃ©-requisito estrutural da navegaÃ§Ã£o clÃnica

Resultado esperado
O app pode carregar fluxo local independentemente da conta conectada.

Fase 5 â€” Migrar os domÃnios principais
Ordem sugerida
1. pacientes
2. sessÃµes
3. prontuÃ¡rios
4. documentos
5. relatÃ³rios
6. agenda

Resultado esperado
O fluxo clÃnico principal passa a funcionar sobre a base local.

Fase 6 â€” Reduzir acoplamentos com infraestrutura remota
Objetivo
Retirar do fluxo principal as adaptaÃ§Ãµes a backend obrigatÃ³rio.

MudanÃ§as
- reduzir dependÃªncia de readiness gate para jornadas principais
- rebaixar wake-up banner a fluxos conectados especÃficos, se ainda existir
- separar conectividade do core clÃnico do estado de integraÃ§Ãµes opcionais

Resultado esperado
O frontend deixa de comunicar â€œservidor indisponÃvelâ€ como bloqueio central do uso clÃnico local.

Refactors concretos recomendados

Refactor 1 â€” Quebrar `pages/Index.tsx`
Criar pelo menos:
- `AppBootstrap`
- `AuthenticatedApp`
- `UnauthenticatedApp`
- `PageResolver`
- `NavigationController`

Refactor 2 â€” Isolar o shell
Criar uma camada clara para:
- sidebar
- header
- frame da aplicaÃ§Ã£o
- navegaÃ§Ã£o principal
- controle de pÃ¡gina atual

Refactor 3 â€” Criar hooks por domÃnio
Exemplos:
- `usePatients()`
- `usePatient(id)`
- `useSessions()`
- `useSession(id)`
- `useRecords()`

- `useDocuments()`

Esses hooks devem consumir services/repositories, nÃ£o `fetch` direto.

Refactor 4 â€” Separar API remota em adapters
Em vez de o domÃnio depender direto de `apiClient`, criar adapters como:
- `patientsRemoteAdapter`
- `sessionsRemoteAdapter`
- `recordsRemoteAdapter`

Refactor 5 â€” Redefinir o papel das stores
Stores devem guardar:
- estado de interface
- estado de navegaÃ§Ã£o transitÃ³ria
- filtros e seleÃ§Ã£o
- feedback visual

Stores nÃ£o devem carregar o papel de base clÃnica primÃ¡ria.

CritÃ©rios de pronto por etapa

Etapa 1 pronta quando
- `Index.tsx` deixar de concentrar a maior parte da orquestraÃ§Ã£o do produto

Etapa 2 pronta quando
- mÃ³dulos principais estiverem organizados por domÃnio

Etapa 3 pronta quando
- pelo menos um domÃnio consumir persistence local via repository

Etapa 4 pronta quando
- auth online estiver separada do carregamento do core local

Etapa 5 pronta quando
- pacientes, sessÃµes e prontuÃ¡rios estiverem rodando em fluxo local-first

Etapa 6 pronta quando
- backend remoto deixar de ser prÃ©-requisito tÃ©cnico do fluxo clÃnico principal

---

Riscos do refactor

1. Refactor estÃ©tico sem ganho estrutural
Mover arquivos sem redefinir responsabilidades nÃ£o resolve o problema.

2. Tentar migrar todos os domÃnios ao mesmo tempo
Isso aumenta risco e dificulta validaÃ§Ã£o incremental.

3. Misturar repository com UI state
Isso pode gerar nova dÃvida estrutural.

4. Tentar resolver sync cedo demais
Sync deve vir depois que o core local estiver estÃ¡vel.

DependÃªncias deste plano
Este plano depende de:
- ADR do banco local
- ADR da criptografia local e backup
- ADR de autenticaÃ§Ã£o local versus conta online
- RFC local-first

Ordem de prioridade real
Prioridade mÃ¡xima
- quebrar `Index.tsx`
- modularizar domÃnios
- introduzir repositories locais
- migrar pacientes e sessÃµes

Prioridade alta
- refactor da autenticaÃ§Ã£o
- migrar prontuÃ¡rios
- isolar adapters remotos

Prioridade mÃ©dia
- migrar documentos e relatÃ³rios
- revisar camada de conectividade remota

Resultado esperado
Ao final deste refactor, o frontend do Ethos deve estar apto a:
- operar o core clÃnico com persistÃªncia local como fonte primÃ¡ria
- tratar o remoto como opcional
- evoluir por domÃnio com menos acoplamento
- sustentar backup, restore e futura cloud opcional sem reescrita total

---

PrÃ³ximos artefatos recomendados
Depois deste plano, os documentos mais Ãºteis sÃ£o:
1. estrutura alvo de pastas do frontend
2. plano de quebra do `Index.tsx`
3. contrato dos repositories por domÃnio
4. plano de migraÃ§Ã£o de pacientes e sessÃµes
5. matriz de dependÃªncias entre mÃ³dulos